

Webview implementation for Android and IOS

☀️ Enhance user experience with our smooth WebView ID verification for Android and iOS.

Integrating the IDVerse ID Verification flow via webview allows you to provide users with an in-app experience. A “webview” approach is when the IDVerse ID Verification flow is loaded through the app so the end user does not need to go to a browser to access the flow.

This page offers technical guidance for implementing the IDVerse ID Verification flow via webview. We recognize there can be multiple ways to implement a technical solution. This page covers some common methods.

If implementing webview, the client is responsible for testing and validating that the flow works as expected and there are no issues. IDVerse performs regular testing of the ID Verification flow for all major browsers. IDVerse is not able to test against each client app and webview integration. Therefore, the client should test thoroughly that the webview integration works and repeat testing with each major client app update and OS update.

To receive the transaction URL you can create an ID Verification transaction via the [Store Transaction](#) API request. Then populate this URL in the webview and load it in the app.

Note:

Please contact your IDVerse representative for the relevant code samples.

Android

There are two possible ways to implement a “webview” in Android.

1. Custom Tabs
2. WebView

Custom Tabs

Note:

Custom Tabs android documentation can be found [here](#).

“Custom Tabs” provides an overlay of the browser, which opens on top of the app. This is faster than redirecting to a separate browser app and prevents the user from leaving the app, creating more of an “in-app” experience for the user.

Description from Android documentation:

Custom Tabs are a feature in Android browsers that gives app developers a way to add a customized browser experience directly within their app.

Loading web content has been a part of mobile apps since the early days of smartphones, but older options can present challenges for developers. Launching the actual browser is a heavy context switch for users that isn't customizable, while WebViews don't support all features of the web platform, don't share state with the browser and add maintenance overhead.

Custom Tabs offer a better user experience than simply opening an external browser. They allow users to remain within the app while browsing, increasing engagement and reducing the risk of users abandoning the app. They accomplish this by being powered directly by the user's preferred browser, and automatically sharing the state and features offered by it. You don't need to write custom code to manage requests, permission grants, or cookie stores.

Note:

IDVerse recommends implementing “webview” using this approach.

Benefits

- Simpler to implement than an embedded WebView.
- The app is not required to set any permission or user agent values.
- As the flow is being displayed within an instance of the browser, there are less likely to be unexpected issues. This is because the IDVerse ID Verification flow is optimized for testing running in all major browsers.

How to implement

1. The app will create the transaction (e.g. via a “[Store Transaction](#)” API request).
2. Set the Custom Tabs URL to the transaction URL.
3. Load the Custom Tab.

WebView

Note:

WebView android documentation can be found [here](#).

This is when the webpage content is loaded directly within the app. It does not appear as a separate browser.

Benefits

- A more seamless experience for the user - they are not redirected to a separate browser app or browser overlay.
- This approach might be preferred as it offers a more complete “in-app” experience.

Considerations

- Requires more complex development to implement.
- IDVerse cannot test the specific webview implementation for the client app. This is the responsibility of the client to test and validate. Testing should be conducted with each major app upgrade and OS update to ensure there are no adverse effects.

Requirements

- Enable javascript in the webview: `webview.settings.javascriptEnabled = true`
- Enable domStorageEnabled in the webview: `webview.settings.domStorageEnabled = true`
- Enable file access in the webview: `webView.settings.allowFileAccess = true`
- Set WebView user agent to the latest Chrome user agent e.g.:
`webview.settings.userAgentString = "Mozilla/5.0 (Linux; Android 10; K) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/95.0.0.0 Mobile Safari/537.36"`
- The app needs to request permissions in the Android Manifest and request permission from the user:
 - `android.permission.CAMERA`
 - `android.permission.ACCESS_NETWORK_STATE`
 - `android.permission.INTERNET`
 - `android.permission.WRITE_EXTERNAL_STORAGE`
 - `android.permission.READ_EXTERNAL_STORAGE`
- Embedded Webview cannot open the camera itself. When WebView is trying to open the camera, it triggers `onShowFileChooser` function in `WebChromeClient`. This function has a callback function that is used to send the captured photo to the WebView. The application should then intentionally open the camera and fetch the URI of the image file. Fetching the URI of the image differs according to the Android Version. (Version 24 or later will require a `FileProvider` to be defined in the `AndroidManifest.xml` file. Earlier versions do not require this. Simply using the `Uri.fromFile` function would be ok).
- Trigger the callback to send the image back to the `WebView`.
- There may be an issue trying to write/read files from the external storage download directory. From Android13 this is not normally allowed. It is suggested to use the cache directory instead:
`var downloadsDir = cacheDir`.

iOS

There are two possible ways to develop webview functionality on iOS.

1. `SFSafariViewController`
2. `WKWebView`

SFSafariViewController

Note:

`SFSafariViewController` apple documentation can be found [here](#).

This causes a simple overlay of the Safari browser to open on top of the app. This is faster than redirecting to a separate browser app and prevents the end user from leaving the app, creating more of an "in-app" experience for the end user.

Note:

IDVerse recommends implementing "webview" using this approach.

Benefits

- Simpler to implement than an embedded `WebView`.
- The app is not required to set any permission or user agent values.
- As the flow is being displayed within an instance of the browser, there are less likely to be unexpected issues. This is because the IDVerse ID Verification flow is optimized for testing running in all major browsers.

How to implement:

1. The app will create the transaction (e.g. via a "[Store Transaction](#)" API request).
2. Set the `SFSafariViewController` URL to the transaction URL.
3. Load the `SFSafariViewController`.

WKWebView

Note:

`WKWebView` apple documentation can be found [here](#).

This is when the webpage content is loaded directly within the app. It does not appear as a separate browser.

Benefits

- A more seamless experience for the user - they are not redirected to a separate browser app or browser overlay.
- This approach might be preferred as it offers a more complete “in-app” experience.

Considerations

- Requires more complex development to implement.
- IDVerse cannot test the specific webview implementation for the client app. This is the responsibility of the client to test and validate. Testing should be conducted with each major app upgrade and OS update to ensure there are no adverse effects.

Requirements

- Allow Javascript. To do this, set `KWebpagePreferences().allowsContentJavaScript = true` .
- Set custom user agent to the latest Chrome browser on iPhone e.g.
`webView.customUserAgent = "Mozilla/5.0 (iPhone; CPU iPhone OS 16_3 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) CriOS/110.0.5481.83 Mobile/15E148 Safari/604.1"` .
- Set `WKWebViewConfiguration.allowsInlineMediaPlayback = true` . If this is not set it will open the video camera using the native full-screen controller. This may cause the on-screen elements to disappear.

We Are Here to Help!

If you encounter an issue, find a bug, or require assistance, please contact our **[Support Desk](#)** to find the solution. Don't forget to provide all the critical information about the issue.

 Updated 3 months ago

[← Journey Initialization](#)

[IDVerse SSO Integration Guide →](#)

Did this page help you?  Yes  No